TNA Interpreter Layer v1

A Domain Mapping Framework for TNA Core Language

0. Purpose

The Interpreter Layer defines how to map:

$$\text{Real System} \to (\Omega, \partial\Omega, R, H_s, \mathcal{F}_B)$$

It enables:

- instantiation of TNA in specific domains
- comparison across domains
- empirical proxy extraction

1. Interpreter Structure

1.1 Mapping Function

$$\mathcal{I} : \mathcal{S} \to \Sigma$$

Where:

- $\mathcal{S}$ = real system
- Σ = TNA system

1.2 Components

```
Interpret(System S):
    define Ω
    define ∂Ω
    define R(x)
    define proxies for H_s
    define proxies for F_B
    return Σ
```

---

2. Mapping Rules

Rule 1 — Domain Identification

$$\Omega := \text{Set of admissible states under system constraints}$$

Examples:

- Physics $\rightarrow$ valid spacetime configurations
- ML $\rightarrow$ parameterized model outputs
- Cognition $\rightarrow$ representable mental states

Rule 2 — Boundary Detection

$$\partial\Omega := \{x \mid R(x) \approx 0\}$$

Operational definition:

- instability
- divergence
- contradiction
- non-generalization

Rule 3 — Reliability Function

$$R(x) := \text{predictive / explanatory success}$$

Proxies:

- Physics → model accuracy
- ML → validation performance
- Cognition → coherence / confidence

Rule 4 — Expansion Proxy

$$H_s := \text{rate of growth of representable states}$$

Proxies:

- scientific output
- model capacity
- conceptual diversity

Rule 5 — Boundary Pressure

$$\mathcal{F}_B := \text{rate of failure at boundary}$$

Proxies:

- anomalies
- unresolved questions
- instability frequency

Rule 6 — Cognitive Coupling (optional)

$$\mathcal{F}_Q := \text{rate of explicit questioning}$$

---

3. Domain Templates

3.1 Physics Interpreter

```
Ω := set of physically valid states
∂Ω := singularities, QM–GR breakdown
R := predictive accuracy of models
H_s := growth of theoretical/experimental space
F_B := anomalies, divergences
```

3.2 Machine Learning Interpreter

```
Ω := model hypothesis space
∂Ω := generalization failure regions
R := validation performance
H_s := model capacity growth
F_B := loss spikes, OOD failure
```

3.3 Cognitive Interpreter

```
Ω := representable mental states
∂Ω := existential / paradox zones
R := coherence / predictive stability
H_s := conceptual expansion
F_B := questioning rate (F_Q)
```

3.4 Complex Systems Interpreter

```
Ω := system configurations
∂Ω := phase transition regions
R := stability metrics
H_s := state-space growth
F_B := instability / bifurcation rate
```

---

4. Execution Pipeline

4.1 Standard Flow

```
Input: System S

1. Define Ω
2. Identify ∂Ω via failure regions
3. Estimate R(x)
4. Compute H_s proxy
5. Compute F_B proxy
6. Analyze transitions

Output: Σ + boundary dynamics
```

---

5. Cross-Domain Comparison

5.1 Structural Equivalence

Two systems S_a, S_b are equivalent if:

$$\mathcal{I}(S_a) \sim \mathcal{I}(S_b)$$

Meaning:

- similar boundary behavior
- similar expansion dynamics

5.2 Mapping Function

$$\phi : \partial\Omega_a \to \partial\Omega_b$$

Constraint:

- preserves structure, not content

---

6. Observable Extraction

6.1 Required Condition

A TNA mapping is valid if:

$$\exists \text{ observable proxies for } R, H_s, \mathcal{F}_B$$

6.2 Testable Relations

T1 — Boundary Coupling

$$\mathcal{F}_B \uparrow \Rightarrow H_s \uparrow$$

T2 — Cognitive Link

$$\mathcal{F}_Q \propto \mathcal{F}_B$$

T3 — Instability Precedes Expansion

$$\text{Instability} \Rightarrow Expand(\Omega)$$

---

7. Failure Modes

F1 — Overextended Domain

Ω defined too broadly $\rightarrow$ no boundary detectable

F2 — Underspecified Ω

Too narrow → artificial boundaries

F3 — Proxy Collapse

No measurable proxies → non-falsifiable mapping

---

8. Minimal Interface (IA-Oriented)

Input Schema

```
System:
  type: [physics | ml | cognition | complex]
  variables: [...]
  constraints: [...]
```

Output Schema

```
Σ:
  Ω: defined
  ∂Ω: detected
  R: function/proxy
  H_s: metric
  F_B: metric
  dynamics: description
```

---

9. Canonical Interpretation Statement

> *A system is fully interpreted when its failures define its boundary, and its boundary defines its evolution.*

---

10. Compression (IA-ready)

Core Interpreter Rule

```
failure → boundary → pressure → expansion
```